

Two Scheduling Algorithms for Two-Processor Systems in Exponential and Near Linear Running Time

By Patrick McGrath, CSCI 429, VIU, Nanaimo, BC

December 8, 2025

Abstract

Minimizing execution times of a set of computational tasks is a fundamental goal of all programming. When the partial order of dependencies of a set of tasks is expressed as a directed acyclic graph, dag and when the system has more than one processor, optimally scheduling the tasks will result in the lowest total runtime the tasks can have. But, optimal schedules must be produced efficiently as well. The two algorithms discussed here are concerned with efficiently producing optimal schedules for two-processor systems. EG Coffman jr and RL Graham developed their *Algorithm A* for this problem in 1972 [1]. In 1976, Ravi Sethi improved the running time of *Algorithm A* with clever implementations in his unnamed linear labeling and scheduling algorithm [2]. Both algorithms follow a pattern of labeling tasks based on the partial order of the tasks they are a prerequisite of, then producing the schedule based on the labels. The correctness of both algorithms covers the optimality of the schedules. Whereas the running times cover the efficiency of producing them. It is shown that Sethi's algorithm has a running time of $O(n\alpha(n)+e)$, where $\alpha(n)$ is a near constant number. And, this is an improvement to *Algorithm A*'s $O(n^2)$.

Terminology and Constraints

The terms used throughout this paper reflect the terminology used by Coffman & Graham, and Sethi in their own papers [1][2]. They each use similar but different terminology. In each section, their respective terminology is used, with minimal changes.

A finite set of tasks, T , is represented as a directed acyclic graph (dag) with the symbols used being G (Coffman & Graham) and D (Sethi). There is a partial order, $<$, on the dag, where each directed edge represents the immediate precedence relationship of tasks. Each task is a node, and can be represented as T_j where j is the label the task eventually receives. Tasks may be denoted with other symbols too. The label of a task is represented by $\alpha(T_j) = j$ (Coffman & Graham), $L(T_j) = j$ (Sethi). If $T < T'$, then T is a predecessor of T' and T' is a successor of T . If there exists no T'' such that $T < T'' < T'$, then T is an immediate predecessor of T' and T' is an immediate successor of T . In this case, in the dag, there will be an edge from T to T' . The set of immediate successors of T is $S(T)$. A terminal task, $T_{terminal}$, is one where $S(T_{terminal}) = \emptyset$. In other

words, they have no immediate successors. Likewise, an initial task is one that is not an immediate successor to any task. In terms of $S(T)$'s, $T_{initial} \notin S(T_j), \forall T_j \in G$.

The algorithms discussed here both operate under the same constraints. All tasks are nonpreemptive, meaning once they commence, they must continue execution uninterrupted until completion. The schedules are produced statically, meaning they are produced prior to executing the first task, with the full dag known ahead of scheduling. No new tasks may be added after the schedule is made. All tasks are equally-weighted, meaning they each take the same amount of time to execute.

The start of an execution time unit is represented by the abstract time unit μ_i , where i is the integer value of the time, starting at 0. The start of an execution time for a given task T is represented by $\tau(T)$. A task is considered ready at any given time unit if all of its predecessors have finished execution. Each processor may execute up to 1 task per time unit. There are two identical processors, $P1$ and $P2$. $P1$ will always have priority if both processors attempt to schedule the same task. Each task is executed once and only once. If a processor is not executing a task at any given time, then it is idle at that time. In this case, it is said to be executing an empty task, denoted by $\emptyset$. The label of an empty task is 0, less than the smallest actual label, 1. An ordering of tasks is represented by $L = (T_n, T_{n-1}, \dots, T_1)$. The total execution time for a given list L is represented by $\omega(L)$.

Coffman & Graham

How it works

Labeling

The following explains the labeling L as described by Coffman and Graham [1]. The first step to schedule the tasks is to label them. Labeling is done bottom-up. All the terminal tasks are labeled first, incrementing from 1. If n is the number of tasks, then one of the initial tasks will eventually be labeled n . After labeling, a decreasing list of integers from n to 1 will be produced. This is used to make the schedules.

The actual process of labeling is as follows; the t terminal tasks are all labeled in any order from 1 to t . Next is how to label all the remaining tasks so as to maintain their partial order. The method Coffman and Graham used is lexicographic ordering [1][2]. This is similar to alphabetic ordering; when comparing two strings of elements, starting from the first position, as long as there are identical values at the same position in both strings, move to the next position. The smaller of the strings is the one which first has a smaller or no value at the same position.

For the labeling routine, at any time, there is a set of tasks where each has all of their successors already labeled. Each of these tasks will have a corresponding decreasing list of their successor's labels. To label the next task, the task with the lexicographically smallest of all of these lists, gets the next label value. Ties are broken in any way.

For example, in the case where just all the t terminal tasks have been labeled and nothing else, the tasks whose sets of successors contain only terminals, will have the decreasing list of labels of those terminals lexicographically compared. The smallest of these

will be labeled $t+1$. After labeling that task, for all of its predecessors, if this task was its last unlabeled successor, that task's decreasing list of successors' labels is added to the set of tasks that need to be labeled in lexicographically smallest order. This is done until the last label is labeled. The decreasing list of labels produced this way is denoted by L^* [1]. It will be shown to be the list that produces an optimal schedule.

Though this labeling routine is from Coffman and Graham's algorithm A, the pseudocode is from a description of it from Sethi's paper, and is included in Fig 1 [2]. An example dag labeled using this algorithm is included in Fig 2 [2].

Routine Labeling L

Given a dag D , assign labels $\alpha(x)$ to each node x in D .

1. A terminal node in D is a node with no edges leaving it. Label 1 is assigned to one of the terminal nodes in D .
2. Suppose labels 1, 2, ..., $j-1$ have been assigned. Let S be the set of nodes x such that all successors of x have been assigned labels. S may be empty. For each node x in S define a list $l(x)$ as follows. Let $y_1, y_2, \dots, y_k$ be all the nodes such that (x, y_i) is an edge for $1 \leq i \leq k$. Then $l(x)$ is the decreasing sequence of integers formed by ordering the set $\{L(y_1), \dots, L(y_k)\}$. Let x be an element of S such that all x' in S , $l(x)$ is lexicographically smaller than $l(x')$ (Break ties at will). Define label $L(x)$ to be j . End

Fig 1. Pseudocode for Coffman & Graham's Labeling Routine [1][2]

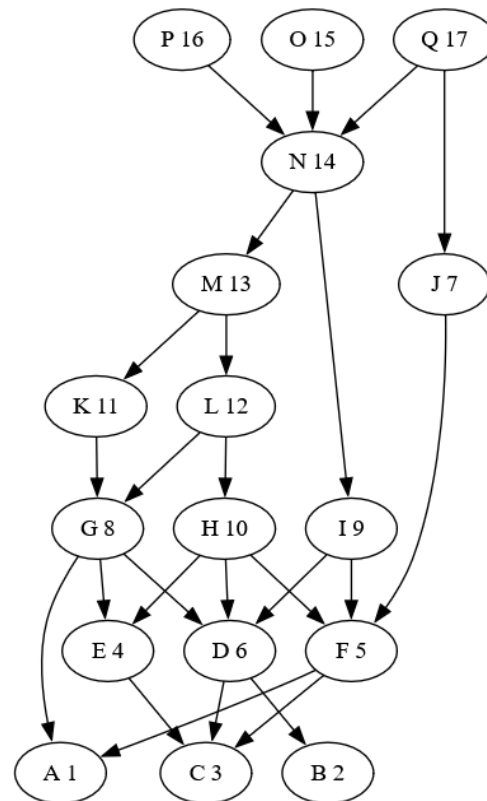


Fig. 2. An example dag, labeled using Routine Labeling. This graph is from Sethi's paper. The lettering represents the names of the tasks. The numbering represents the corresponding labeling. $L^* = (17, 16, \dots, 1)$ [2]

This method of labeling ensures that tasks with higher precedence have higher labels. Furthermore, at any position in the list L^* , the task corresponding to the label at that position, will have all of its predecessors' labels be prior, with higher label values. And similarly, all of its

successors' labels will come after, with lesser labels. This implies that if at any time we ignore all already executed tasks in the list, the “top” of the list will always be a ready task to be executed.

This observation is the basis for *Lemma 1* [1]. During scheduling, $P1$ will always have a highest labeled ready task to execute. Therefore, $P1$ is never idle from μ_0 to $\omega(L^*)$.

Scheduling

After labeling the tasks, the labels are used to produce the schedule, as described by Coffman and Graham [1]. Coffman and Graham describe that the processor instantaneously scans the list L^* to find the highest labeled ready task [1][2]. $P1$ breaks ties and takes the highest and $P2$ continues scanning. $P2$ may not find a ready task and therefore be idle during that time. The actual implementation can greatly change the running time.

One possible method is to keep two arrays for information about the tasks. The first array is of task readiness, $ready[]$, and the second array is for counting the unexecuted immediate predecessors of that task, $pred[]$. Initialize $pred[]$ by breadth-first-search. On each edge (T_j, T_k) , increment $pred[T_k]$ by one. Initialize $ready[]$ so that all tasks are not ready, except the ones where $pred[T_j]=0$. After initialization, in order to schedule, at each time unit, $P1$ will schedule the task T_j where $ready[T_j] = ready$ and T_j is the highest labeled of all such tasks. If there is a second one T_{j-k} , $ready[T_{j-k}] = ready$, k is minimal, $P2$ can schedule that one. Then, for each successor of T_j , $T_{j-q} \in S(T_j)$, decrement their unexecuted predecessor count, $pred[T_{j-q}] = pred[T_{j-q}] - 1$. If any $pred[T_{j-q}] = 0$, set that task to ready, $ready[T_{j-q}] = ready$. Do the same for $S(T_{j-k})$. Also, set the just scheduled tasks to unready, $ready[T_j] = ready[T_{j-k}] = unready$, so they will not be executed twice. Repeat until all tasks have been scheduled.

This method should not increase the running time. The breadth-first-search is in $O(e)$, which is $O(n^2)$ in the worst-case. Each scan for a ready task is in the order of n , $O(n)$. But also, the successors of the task actually executed need their $pred[]$ counts decremented. This is also of the order n , $O(n)$. Combining these times, we get $O(n^2) + O(n) + O(n) = O(n^2)$.

The pseudocode given in *Fig 3* is from Sethi's description of Coffman and Graham's *algorithm A* and not from the above example method [2]. An example schedule produced using the dag in *Fig 2* is provided in *Fig 4* [2]. The other information on *Fig 4* will be explained later.

Routine Scheduling

Whenever a processor becomes available, the list L is scanned from left to right. The first unexecuted task that is ready to execute is assigned to the processor.

Fig 3. Pseudocode for Coffman & Graham's Scheduling Routine. [2]

μ	0	1	2	3	4	5	6	7	8
χ_k	4		3	2	1				0
$P1$	Q 17	O 15	N 14	M 13	L 12	H 10	D 6	E 4	C 3
$P2$	P 16	J 7	$\emptyset$	I 9	K 11	G 8	F 5	B 2	A 1
n_k	2		1	1	4				1

Fig. 4 The schedule of the graph from Fig. 2. Execution goes from left to right, starting at μ_1 . The V_k 's are in orange. The W_k 's are in green. The χ_k 's are bold outline-lined. [2]

Correctness

Next is to show that using this scheduling method on the list just produced by labeling, L^* , will result in a schedule with the shortest possible execution time of all schedules built by all lists using the same scheduling method on the same two processors.

This is stated in *Theorem 1*, $\omega(L^*) \leq \omega(L)$, $\forall L$ [1]. The proof for *Theorem 1* involves analyzing the schedule produced by defining “blocks” of tasks in the schedule. Each of these blocks must be executed before the next one can begin, with no units of time in between. Then, the proof shows that the total running time of these blocks is the smallest possible running time. The proof is built-up from smaller components. This section of this paper intends to cover the components and their purposes.

For the proof of *Theorem 1*, there are a few definitions needed [1]. The blocks are represented by χ_k^1 , with k being enumerated from the end to the beginning, starting at 0 and ending at m . The χ_k blocks have corresponding tasks, W_k and V_k , where their execution times

are both the last time unit in their blocks, ie $\tau(W_k) = \tau(V_k) = \left(\sum_{p=m}^k \omega(\chi_p) \right) - 1$.

It should be noted that none of the terminology used in this theorem that was not already introduced in the labeling and scheduling sections, are used for labeling or scheduling. They are only used for proving optimality.

W_k and V_k are defined recursively from the end of the execution time. V_0 is the last task that $P1$ will execute. V_0 ends at $\omega(L^*)$. Therefore, $\tau(V_0) = \omega(L^*) - 1$. W_0 is the last task that $P2$ will execute. It may be an empty task. For higher W_k 's, each W_k is defined as a task T , with three requirements. First, its label must be less than that of the subsequent V_{k-1} 's label, ie $\alpha(T) < \alpha(V_{k-1})$. Second, the execution time of that task must be earlier than that of the subsequent V_{k-1} , ie $\tau(T) < \tau(V_{k-1})$. And third, the execution time of this T must be maximal, i.e. higher than all other T 's that satisfy the first two requirements. Since $P1$ must execute higher labeled ready tasks, and this W_k has a lower label than a V_{k-1} that $P1$ will subsequently execute, $P2$ must execute W_k . Finally, V_k is simply defined as the task that executes at the same time as W_k , i.e. $\tau(W_k) = \tau(V_k)$.

One way of understanding W_k , would be to consider a naive approach, where at each time unit $P1$ still executes the current highest label. But, $P2$ only executes the next highest if it's

¹ Lower-case Greek “Chi.”

ready, idling otherwise without checking for lower tasks' readiness and potentially leaving lower labeled ready tasks waiting. If there were lower labeled ready tasks, each W_k is either an empty task or one of those ready tasks as they are instead scheduled to be executed at that time in L^* .

Since the empty task is labeled lower than all actual tasks, it will always be a W_k . Therefore, if no W_1 is found, there can be no idling, prior to potentially the last time unit, for odd sized task sets. Therefore, the schedule is trivially optimal [1]. The proof involves showing that even if you find W_1 , the schedule is still optimal.

For this, we need to define the contents of the χ_k blocks [1]. If we have W_k 's, where $0 \leq k \leq m$, then there are m W 's, V 's and χ 's. Each χ_k is a set of tasks T such that all their execution times are after the previous V_{k+1} , and are prior to or at the same time as the next V_k , i.e. $\tau(V_{k+1}) < \tau(T) \leq \tau(V_k)$, $T \in \chi_k$. The exception is W_k . They are the only tasks not present in χ_k . Also, since there is no V_{m+1} , χ_m is the set of tasks that execute at or before $\tau(V_m)$, again excluding W_m .

The length of execution time of each χ_k is denoted by n_k [1]. The size of each χ_k is equal to $2n_k - 1$. Because, there are two processors and one excluded W_k .

By now, the extra information in Fig 4 should be clear. The "heart of the proof," as it is described by Coffman and Graham, is proving that for any task T in a block χ_k , and a second task T' in its predecessor block χ_{k+1} , T' will be before T in the partial order, i.e. if $T \in \chi_k$ and $T' \in \chi_{k+1}$ then $T' < T$ [1]. This is proven by double-induction because T can be any task in χ_k , not just a certain one. With the same being true for T' and χ_{k+1} .

The proof works by having a first induction showing that for any task T minimally timed within its χ_k , that is $T \in \chi_k$ and $\tau(T) = \tau(V_{k+1}) + 1$, V_{k+1} in the previous χ_{k+1} , will be before T in the partial order, i.e. $V_{k+1} < T$. This proof is done by showing that if there was a $T' < T$ at $\tau(V_{k+1})$, that is one time unit prior to $\tau(T)$, then it can only be V_{k+1} . And since, $T' < T$ and $T' = V_{k+1}$ then $V_{k+1} < T$.

The second part of the first induction shows that this is also true when $\tau(T)$ is not minimal, but T is still in χ_k . That is when it is not in the first time unit of χ_k , V_{k+1} still needs to finish executing before T can start. This is done using a similar proof as in the previous step.

The second induction shows that for any other $T' \in \chi_{k+1}$, $T' < T$. It considers that since V_{k+1} has already been shown to need to finish execution before any $T \in \chi_k$ can commence and inductively assumes that for any other task $T' \in \chi_{k+1}$, all subsequent tasks $T'' \in \chi_{k+1}$ have already shown that $T'' < T$. And by the transitivity of $<$, $T' < T$ as well. This situation, where T has successors in χ_k , is considered trivial, as $T'' = V_{k+1}$ would be considered a base case, in addition to the inductive reasoning.

So, it must be considered when $S(T') \cap \chi_{k+1} = \emptyset$, that is when T' has no successors in its own block. The proof then shows that in this case too, $T' \in \chi_{k+1} < T \in \chi_k$ [1]. For this, we will denote the set of tasks in χ_k that have no predecessors in χ_k with a new term, I_k . The proof involves showing that if $T' \in \chi_{k+1}$ has no successors in χ_{k+1} , then $S(T') \cap \chi_k = I_k$. This means that any successor of T' that is in χ_k cannot have a predecessor also in χ_k . This implies that the remaining tasks, $\chi_k - I_k$, have predecessors in χ_k and that they are not successors of T' . Those predecessors in χ_k will either be in I_k or be successor of I_k . Due to the transitivity of $<$, $T' \in \chi_{k+1} < T'' \in I_k < T \in (\chi_k - I_k)$. Simplified, we get $T' \in \chi_{k+1} < T \in (\chi_k - I_k)$. And since $\chi_k - I_k + I_k = \chi_k$, $T' \in \chi_{k+1} < T \in \chi_k$, $0 \leq k \leq m$, as desired.

At this point in the proof, it has been shown that if $T \in \chi_k$ and $T' \in \chi_{k+1}$, then $T' < T$ [1]. In other words, each block must be completed before the next block can start. Regardless of the

list used to devise a schedule, these blocks will exist with no gaps between them. Therefore, to have an optimal schedule, it must be shown that by following the scheduling algorithm on the list L^* made by the labeling algorithm, the blocks produced will be minimally sized.

Since there are no gaps between the χ_k 's, the total running time of the graph G is at least the sum of the running times of the χ_k 's [1]. Recall that χ_k has $2n_k - 1$ tasks, requiring n_k units of

time to execute. Therefore G will require at least $\sum_{k=0}^m n_k$

units of time to complete, regardless of the list L used.

And since the length of execution of L^* is exactly that minimum sum, it is therefore the optimal list to use to

schedule, i.e. $\omega(L^*) = \sum_{k=0}^m n_k$, $\omega(L^*) \leq \omega(L)$, $\forall L$.

This concludes the proof of *Theorem 1* [1]. At this point it is shown that Coffman and Graham's scheduling routine in *Algorithm A* can take a decreasing list of labels of corresponding tasks, and produce a schedule out of them for a two-processor system. If that list was produced using the labeling routine in *Algorithm A*, then the schedule produced will have the shortest or tied for shortest total execution time among all possible schedules made this way. Next is the efficiency of *Algorithm A*.

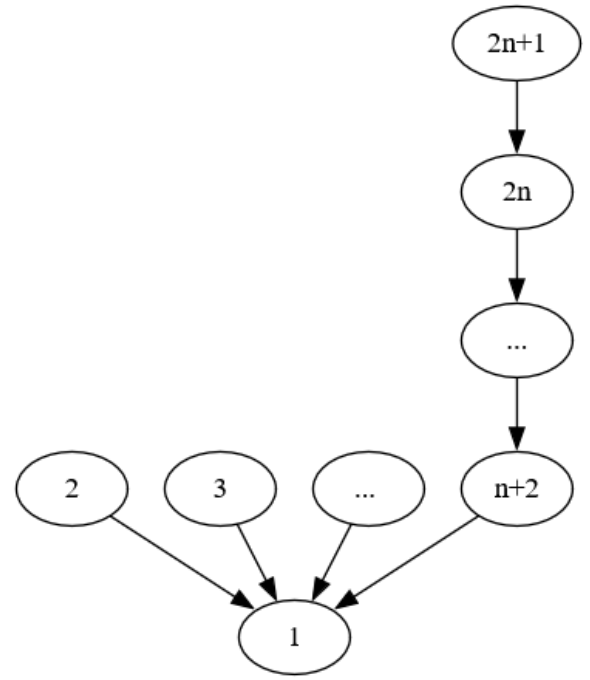


Fig. 5. Example of a labeled graph, from Sethi, that would be inefficient to schedule using Coffman and Graham's Algorithm A. [2]

Running Time

The analysis of the running time of *Algorithm A* is fairly rudimentary. First, the labeling routine. For each task, their lexicographical list could have a length in the order of n , meaning the number of comparisons that could be needed to label a single node is also in the order of n . The labeling itself is in constant time. There are n tasks to be assigned labels. Therefore the labeling routine has a running time of $O(n^2)$ [1].

For the scheduling routine, Coffman and Graham's *Algorithm A* explains the steps for assigning tasks to processors [1]. But it is described as instantaneous, which does not help explain its running time. Earlier, a proposed implementation that follows the same logic was given. Its running time is also $O(n^2)$. This further confirms the claimed overall running time of $O(n^2)$. This running time is worse than near-linear. But it is better than prior algorithms that solve the same problem. For instance, Fuji (1969/1971) has a running time of $O(n^4)$ [3][4].

Sethi provides an example of a labeled dag, where the L^* would be inefficient, $O(n^2)$, to schedule [2]. This example is *Fig 5*. It should be clear that $P1$ will end up executing the vertical series of tasks from top to bottom. While $P2$ will scan through 2, 3, ..., $n-1$ to get to task n .

Sethi

Sethi's algorithm improves the running time of *Algorithm A* [2]. This is done in both the labeling and scheduling steps. The labeling is improved by replacing lexicographic ordering with a concept of placing tasks in "levels," keeping stacks of edges for those levels and labeling based on a subset of those edges. Scheduling is improved by successively scheduling the labels from highest to lowest. This is done by merging disjoint sets of time units, each identified by their first subsequent time unit that a processor is free, including their own time unit. These improvements will decrease the running time to $O(n\alpha(n)+e)$, where $\alpha(n)$ is the inverse of Ackermann's function, also known as lg^* . Since $\alpha(n)$ is effectively linear, the running time is effectively $O(n+e)$. Note, Sethi uses $\alpha(n)$ for this running time and not for labeling of a task like in Coffman and Graham. For that, he uses $L(x)$ [1][2].

Labeling

First, like with Coffman and Graham, is to label the graph. Sethi's method of labeling produces the same labels as Coffman and Graham, just in an improved running time [2]. The correctness of this algorithm's labeling portion just needs to show that it does indeed produce the same labels as Coffman and Graham. Then, the correctness of those labels have already been proven by Coffman and Graham.

Labeling is done by first placing each task in a "level" [2]. Levels are recursively defined from the set of terminal tasks, where level 1 is the set of terminal tasks. All other tasks are in level i , where $i = 1$ plus the maximum level of all immediate successors. Each level is essentially one plus the furthest distance possible to a terminal node. This ensures that all edges go from one level to another. The method of determining levels is not provided. But, it can be done in $O(n+e)$. This is because each node will receive one level and assigning levels will visit each edge once. In the routine *Label*, as shown in *Fig 6*, this is Step 1.

Next, for each level there is a stack, $EDGES(i)$, of edges leaving tasks at level i [2]. The stacks are initialized empty. Edges (x,y) are only added to $EDGES(level_x)$ when y is labeled in its lower level. Since the tasks are labeled lowest to highest, and the edges are added as the lower leveled tasks are labeled, the top of the stacks will have edges with higher labeled destination nodes than the destination nodes in edges lower in the same stack. This means that instead of doing lexicographic ordering of the full list of labels of successor nodes, only the edges that differentiate labeling of tasks at any given level are needed to label those tasks. This also shows why *Lemma 1* is correct. It states that if x is at a lower level than y , $L(x) < L(y)$ [2]. This is because the level of y will not begin labeling until all lower levels are done.

Routine Label

Start with a dag D and label it according to labeling L . Use a series of stacks of edges, one stack per level i to keep track of the edges leaving level i , $EDGES(i)$.

1. Determine all the levels in D .
2. Since all nodes at level 1 are terminals, $EDGES(1) = \emptyset$. Since no labels have been assigned, j , the lowest label to be assigned is 1. Do Step 3 with $j=1$, $i=1$, $EDGES(i)=\emptyset$.
3. Do Routine Partition to determine all labels for nodes at level i . Do Step 3.1 for all nodes y at level i , in order of increasing label
 - 3.1. Scan the edges entering node y . For all (x,y) , node x at level k , push (x,y) onto $EDGES(k)$.
4. Repeat Step 3 until all levels have been labeled.

□

Fig 6. Pseudocode for Sethi's Routine Label. Labeling L here refers to Coffman and Graham's labeling. [2]

Routine Partition

This routine labels nodes at a given level i . It has three inputs;

- A. A level, i ,
 - B. $EDGES(i)$, a stack of edges leaving level i ,
 - C. j , the lowest label to be assigned at level i .
1. Construct a new node w , such that w becomes the immediate successor of all nodes x at level i . Call w the *tree successor* of x in order to distinguish it from the nodes in the dag D . Let $LOW_LABEL(w) = j$.
 2. While $EDGES(i)$ is nonempty, do Step 2.1. Otherwise, do Step 3
 - 2.1. Let the edge on top of $EDGES(i)$ have y as the destination node. Do Step 2.1.1 followed by 2.1.2 with y as the destination node.
 - 2.1.1. While the edge on top of $EDGES(i)$ has destination y , do Step 2.1.1.1. Otherwise, do Step 2.1.2.
 - 2.1.1.1. Let the edge on $EDGES(i)$ be (x,y) . Let w be the tree successor of x . Enter x into a temporary list for node w . Pop (x,y) from the stack $EDGES(i)$. End 2.1.1.1.
 - 2.1.2. For all nodes w such that Step 2.1.1.1 places a node in the temporary list for w , construct a new auxiliary node v . Node v becomes the new tree successor of all nodes x in the temporary list for w . The edge (x,w) is deleted. If there are now k nodes left with w as their tree successor, then $LOW_LABEL(v) = LOW_LABEL(w) + k$.
 3. Visit the nodes w that have been created in Steps 1 and 2, in any order. If $j = LOW_LABEL(w)$, and there are $k \geq 1$ nodes with w as their tree successor, then assign labels $j, j+1, \dots, j+k-1$ in any order.

RETURN. □

Fig 7. Pseudocode for Sethi's Routine Partition. This routine is called by Routine Label, and ironically does the actual labeling. [2]

The linear labeling algorithm uses two routines, *Label* and *Partition*, Fig 6 and 7 respectively [2]. *Label* determines levels, then repeatedly calls *Partition* to label each level from level 1 up. When *Partition* first gets called on level 1, the t terminals will eventually all receive

random labels 1 to t using the same method that will be used on upper levels. Since labeling the terminals is trivial, Sethi considers the inductive step where levels 1 and 2 have been labeled in his example graph. Fig 8. depicts his example graph, the same one in Fig 2., except with the levels labeled. For his inductive step, he considers when only levels 1 and 2 have been labeled. Fig 9. also depicts this situation and the full labeling of level 3 in the same graph.

At this point, we will make a set of tasks to be labeled, S , populated entirely with the next level, level 3, $S=\{G,H,I,J\}$. This is an equivalence class of nodes [2]. They are considered to all have the same immediate successor, w , the *tree successor* of each node in S , with a value of one plus the highest label assigned so far. Like in Coffman and Graham, we will label these tasks based on the decreasing order of their successor's labels. At this point, the highest label so far is $L(D) = 6$. Therefore the next label assigned will be 7. $EDGES(3)$ will have been populated with edges as nodes A,B,C,D,E,F were labeled, in that order. Let (x,y) be the edge on top of $EDGES(3)$. Create a new set, S' . While the edge on top of the stack is (x_i,y) , move x_i from S to S' , pop the edge. When the top of the stack is an edge (x_j,q) , $q \neq y$, we update y to be q for splitting the sets S and S' . We also need to note how many nodes were not moved into S' . Let k be the number of nodes not moved this way. S' will all point to a new auxiliary node, (v) , with a value equal to the value of S 's *tree successor* node plus k . Then, repeat traversing the top of the stack while the top is (x_i,y) . Split S' into S'' and S' using the same method used to split S into S' and S . This is done until all tasks are in their own sets, or there are no more edges on the stack. Assign tasks in the sets from first the *tree successor*, then the lowest auxiliary node to highest. If the task is alone in their set, assign it that auxiliary node's value. If they are not alone, randomly assign one the auxiliary node's value, j , and the rest receive randomly $j+1$, $j+2$, etc. After assigning a label to task x , before assigning the next, for each edge (u,x) , add that edge to $EDGES(level_u)$. This will ensure the same conditions for the next steps as this step started with and therefore this method can inductively label all the nodes in the graph.

The paper does not explicitly note, because it does not occur in the example, that if at some step of splitting S into S' and S , if multiple tasks remain in S , the method still further splits both S and S' by the same method, with the same next y on the stack.

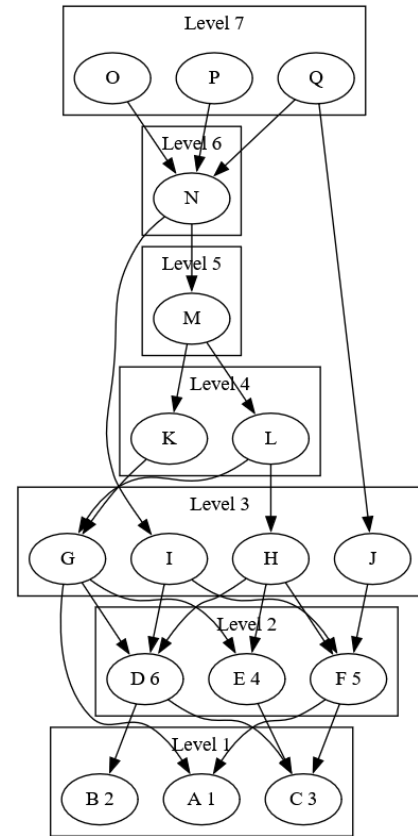


Fig 8. The same graph from Fig 2. Levels have been assigned, and tasks in levels 1 and 2 have been assigned labels. [2]

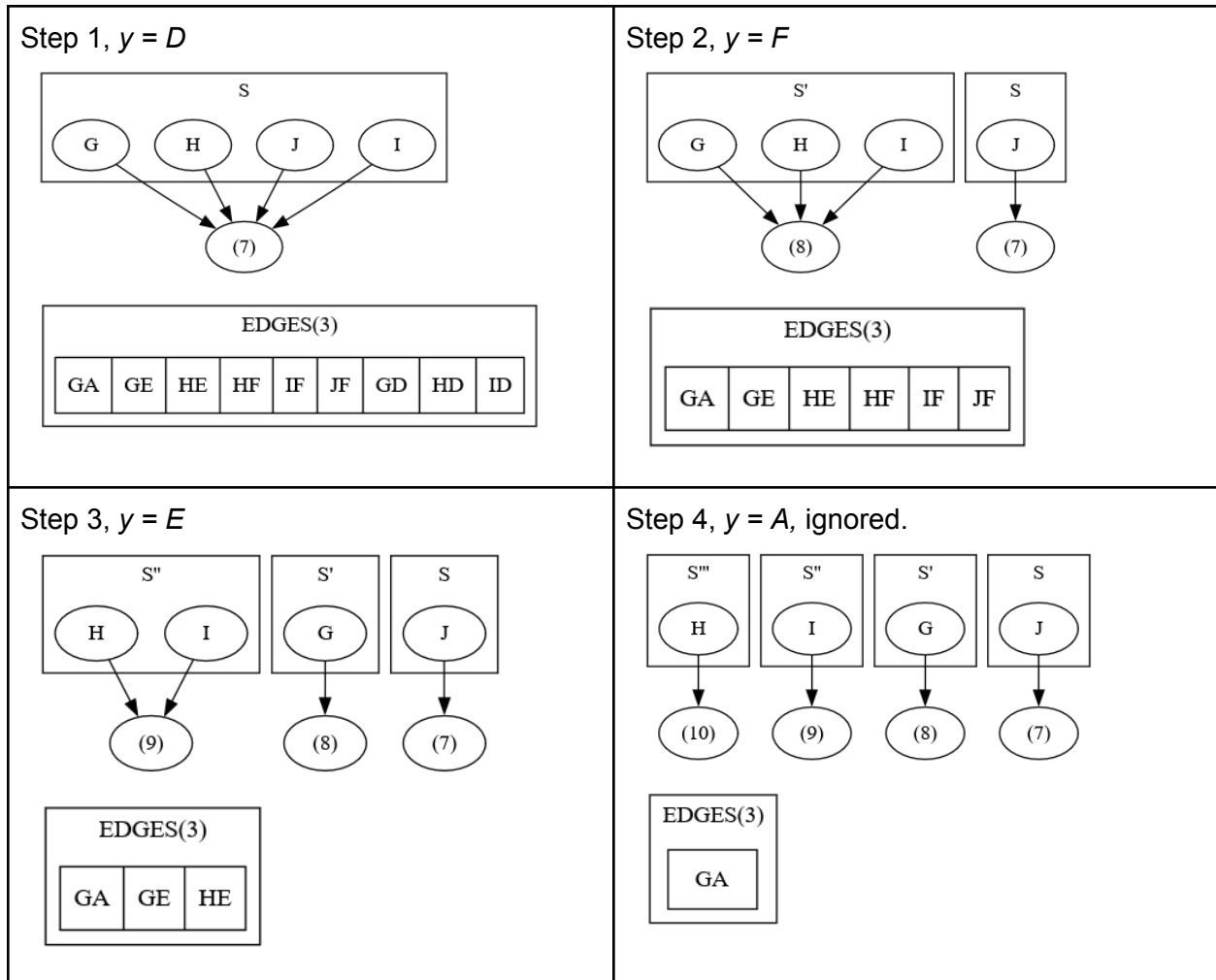


Fig 9. The labeling of level 3 of the graph in Fig 2. The contents of EDGES(3) is shown, with the top of the stack being on the right. Steps 1 to 4 show the labeling of level 3 in the same graph. [2]

Correctness

Now the method for labeling tasks has been shown. But it still needs to be proven that the method produces the same labels as in Coffman and Graham. This is the purpose of *Theorem 1* [2]. The proof requires two new definitions. First, if $x_1, x_2, \dots, x_k$ all point to the same auxiliary node, (v) , then those tasks will eventually be assigned $v, v+1, \dots, v+k$, in some order. The first definition, $\sigma(x)$ denotes the set of tasks, y , such that (x,y) is an edge. The second definition, S denotes the set of tasks, y , that have been processed already. This S is not the same as the S used in partitioning. That previous S is a set of tasks with the same auxiliary successor node. This S is a set of tasks for which all those previous tasks, $x \in (v)$, have an equivalent relationship to tasks, $y \in S$, in the partial order of the graph.

The proof follows that, as a base case, both methods randomly assign the t terminal nodes 1 to t . Then to show that the remaining nodes are labeled the same way, the intersections, $S \cap \sigma(x_i)$ are claimed to be identical for all $x_i \in (v)$ [2]. In other words, if we limit the

set of labeled tasks to just the ones where they are destinations of edges already considered for partitioning S into S and S' , then all $x_i \in (v)$ will have the same connections to those tasks. This is analogous to lexicographic ordering; until a successor task differentiates the labeling of a predecessor task, all predecessor tasks will remain undifferentiated.

The paper does not offer a direct proof of this claim, instead stating it could be proven by induction on the number of execution steps [2]. Another way of thinking about it is by contradiction. Let there be some task $x_q \in (v)$. $\sigma(x_q)$ implies that there is some edge (x_q, z) where z was previously labeled. For the contradiction, if we assume that $L(z) < L(y)$ and this edge does differentiate x_q from the rest of $x_i \in (v)$, then x_q could not have been pointing to the auxiliary node (v) at that time. It would have been removed earlier by partitioning of S when $y=z$.

Since this relationship, $S \cap \sigma(x_i) = S \cap \sigma(x_j)$, $\forall x_i, x_j \in (v)$, is true at all times, and it behaves analogous to lexicographic ordering, the labeling Sethi's *Routines Label* and *Partition* will produce is identical to that of Coffman and Graham's *Labeling L* [1][2].

Scheduling

Now that the method for labeling tasks is established and shown to be correct, we need a method for scheduling [2]. The basis for the scheduling algorithm is two-fold. First, each task takes exactly 1 time unit to execute. And second, since in the list, for any task, its predecessors appear before it and its successors appear after it, the algorithm will see predecessors before it sees successors. From this, schedules can be made successively by making a schedule for (T_n) , then one for (T_n, T_{n-1}) , etc. This can be done until the whole list is scheduled.

The method is covered in *Lemma 2* [2]. If tasks $T_n, T_{n-1}, \dots, T_{i+1}$ have been scheduled, then all of T_i 's predecessors are scheduled. Let τ denote the first time unit that all of these predecessors have been executed, and let u denote the first time at or after τ , where there is a free processor. Schedule T_i at u on one the free processor. *Fig 10* shows the pseudocode for scheduling provided by Sethi [2].

Routine Schedule

Let D be a dag labeled according to labeling L . This routine constructs a list schedule for the task system represented by D using the list of tasks in order of decreasing label.

By way of data structures we need a set for each time unit. Initially $SET(\tau) = \{\tau\}$ for each time unit τ .

1. All initial tasks in D are said to be *ready* at time unit 1
2. For all tasks x , do Step 3 in order of decreasing label.
3. Consider task x . Let x be ready at time unit τ . If τ is in $SET(u)$, then assign x to a processor during time unit u . If u now has no free processors, look at time unit $u+1$. Let $u+1$ be in $SET(v)$. Merge $SET(u)$ and $SET(v)$, calling the new set $SET(v)$. Examine all y such that y is an immediate successor of x . If all predecessors of y have now been assigned, then task y is said to become ready at $u+1$.

□

Fig 10. Pseudocode for Sethi's Routine Schedule. [2]

Correctness

To prove this routine will produce an optimal schedule in the same manner that Coffman and Graham's algorithm does, *Lemma 2* needs to be proven [1][2]. By definition of the lemma, T_i will not be ready at any time prior to τ . And thus the schedules S_{i+1} and S_i will be identical from time units $1, 2, \dots, \tau-1$, with regards to T_i . Then to schedule T_i , at or after τ , it will be at the first time unit when a processor is free and no other task may be scheduled ahead of it. That is, if T_i 's time unit u is the same as another ready task T_j , then $L(T_i) > L(T_j)$ in order for T_i to be scheduled. This is logically the first time at or after τ when a processor is free. This is done for all the remaining tasks until all have been scheduled. This proves that the schedules can be produced successively as stated *Lemma 2*.

Next, *Lemma 3* is used to show the structure of the time unit sets, $SET(u)$, used in *Routine Schedule* [2]. It simply states that for any $\tau \in SET(u)$, u will be the first time unit with a free processor, $u \geq \tau$. The proof is implied by exercise. When scheduling a task at a given time unit, u , if that task occupies the last free processor, then the first time unit at or after u that will have a free processor is at least $u+1$. If both processors at $u+1$ are also occupied, then the first free time unit at or after $u+1$ is at least $u+2$, and transitively u 's first free time is at least $u+2$. Since, initially all time units, τ , will be in their own $SET(\tau)$, if u were the first instance of both processors being occupied, then $SET(u)$ would be merged into $SET(u+1)$. For higher order merges, if u is the time unit that has just had its processors both occupied, then merge $SET(u)$ with the set returned from a $find(u+1)$ function, if implemented using disjoint sets. This $find(u+1)$ should return τ , where $u+1 \in SET(\tau)$.

Bringing this together, we can prove *Theorem 3* [2]. This theorem is for tying together *Lemma's 2* and *3* to show that Sethi's algorithm will, given the same decreasing list of labels, produce the same schedule that Coffman and Graham's algorithm would. Since we already know that the list of labels produced is identical to one produced by Coffman and Graham's *Algorithm A*, the whole of Sethi's labeling and scheduling algorithm is proven correct.

Running Time

Label

Theorem 2 covers the running time of the *Routines Label* and *Partition*, *Fig 6* and *7* [2]. The theorem states that these routines together will take $O(n+e)$ steps to complete. First, assigning levels, Step 1, takes $O(n+e)$ time, as was already shown in the section *Sethi - Labeling* of this paper. Then, Step 2 is in $O(1)$ for a constant number of assignments. Steps 2 and 4 repeatedly call *Partition*, once per level, until all n nodes have received 1 label each. If we attribute Step 2's initial *Partition* call to Step 4, then Step 4 has a running time of $O(n + O(Partition))$. Before analyzing *Partition*, the running time so far is $O(n + e + O(Partition))$.

Next is the running time of *Partition*. Step 2.1.2 in *Fig 7* introduces auxiliary nodes and edges to them [2]. An auxiliary node is only made if there is an edge to a lower level, from a node that is in the current level. When an auxiliary node is visited, the visit can be charged to the edge that caused the visit. For example, in *Fig 9*, Step 2, the creation of (8) and the pointing each of I , H and G to (8) is charged to the edges (I,D) , (H,D) and (G,D) respectively. This

bounds the running time of handling auxiliary nodes by the number edges in the graph. The cost of labeling the nodes has been accounted for in the running time of *Label*. Therefore, *Partition* has a running time of $O(e)$.

To prove *Theorem 2*, we combine these running times. Since *Label* calls *Partition*, the total running time of both is the running time of *Label*, $O(n + e + O(Partition)) = O(n+e)$, as stated by the theorem [2].

Schedule

The proof of the running time of *Schedule* is dependent on the method used to merge the disjoint time unit sets, $SET(u)$. Sethi proposes using either Tarjan or Hopcroft and Ullman's method for handling this problem [2][5][6]. Both methods use amortized costs on merging disjoint sets using path compression, by giving each time unit an initial bank of credits to use to pay for the operations of merging.

Each $SET(u)$ is represented as a rooted tree, where u is the root. Each node can have multiple children, and one parent. Except for the root; it has no parent. The amortized cost comes from the path compression used in the $find(u)$ operations [5][6]. In each $find(u)$, the result is the root of the tree containing u . The node u can be found in constant time. To find the root, a walk from u to parent (to parent) to root is needed. At each step, that node is noted, until the root is found. When the root is found, all those noted nodes have their parents updated to the found root. This has the effect of "flattening" the tree, and making subsequent $find(u)$'s more efficient. This would be expensive if the cost of these operations could not be paid for by the initial credits placed on each time unit. This can be done because the number of repointings of all nodes to new parents because of all merges is bound by a near constant number. The limit of merges is bound by the number of tasks, n , as in the worst case *P2* never gets used.

Tarjan refers to this running time by a near constant number $\alpha(m,n)$, where $m \geq n$ is a number of $find(u)$'s and n is one plus the number of *merge*'s [5]. Since in Sethi's algorithm, each *merge* uses $find(u)$ once and $find(u)$ is not used elsewhere, $m=n$, and $\alpha(m,n) = \alpha(n,n)$ or $\alpha(n)$ for short [2].

Tarjan proves the value of $\alpha(m,n)$ to be $O(m \lg^* n)$ [5]. The term $\lg^* n$ is defined as the number of times needed to recursively apply $\log_2(n)$ until $n \leq 1$. Not to be confused with $\alpha(n)$ as the label of n . If $\alpha(n) \geq 5$, then $n \geq 2^{65536}$. According to *LiveScience.com*, there are roughly 10^{82} atoms in the observable universe [7]. The ratio of $10^{82}/2^{65536}$ is zero followed by roughly 19,000 leading 0's before a digit. In other words, that is a value that is effectively 0. Therefore, as long as n is a number that is no more than astronomically large, $\alpha(n) \leq 5$. By placing a constant 7 credits on each initial time unit; 2 for noting the occupancy on processors and 5 for the near constant cost of merging sets, the cost of the find operations is bound by $\alpha(n)$. Then by noting that the number of merge operations is bound by the number of nodes, the total cost of finding and merging is $O(n\alpha(n))$. This confirms Sethi's claim that scheduling would be done in $O(n\alpha(n))$ [2].

The total running time of Sethi's algorithm is equal to the running time of the labeling portion and the running time of the scheduling portion. Therefore the total running time is near linear, $O(n\alpha(n)+e)$ as claimed [2].

Discussion

Scheduling algorithms exist in a large taxonomy depending on their constraints and applications. Kwok and Ahmad offer an overview of this taxonomy [8]. According to them, finding a schedule for a dag is generally an NP-complete problem. This means, in general finding a schedule to some scheduling problem cannot be done in polynomial-time. But verifying a solution can. However, they claim that there are at least three specific cases where scheduling a dag can be done in polynomial-time. The first is a uniform node-weight free-tree on an arbitrary number of processors. This can be done in linear time, Hu (1961) [9]. The second is when there are exactly two processors and the tasks are equally-weighted. This is the situation discussed in this paper. The third is when there are an arbitrary number of processors, the tasks are equally-weighted and the tasks are interval-ordered, meaning for every edge, the two nodes can be mapped to non-overlapping intervals. Papdimitriou and Yannakakis (1979) presented a linear time algorithm for this problem [10].

Both the algorithms discussed in this paper are interesting but not used practically. The algorithms are for two-processor systems and are from 1972 and 1976. According to Techspot.com, the first multi-processor consumer desktop was not released until IBM's *Power4* in 2001 [11].

Although also not used as a practical scheduling algorithm, Kwok and Ahmad's Parallel Genetic Scheduling algorithm (1997) is used as a benchmark for testing other theoretical scheduling algorithms against [12]. This algorithm belongs to a class of algorithms known as genetic algorithms [8]. These algorithms treat schedules like genomes. There are many genomes in the population. Some process of introducing mutations and exchanging "genetic" information is done on the population. The results of all the individuals are compared to some goal. Some amount of unacceptable individuals are removed. The remainder undergo the same mutations and reduplication technique. This is repeated some number of times prescribed by the algorithm itself. Kwok and Ahmad prove that their algorithm can find an optimal schedule in more than half of all attempts [12]. The running time is too complex to cover here.

References

1. Coffman, E. G., & Graham, R. L. (1972). Optimal scheduling for two-processor systems. *Acta Informatica*, 1(3), 200–213. <https://doi.org/10.1007/bf00288685>
2. Sethi, R. (1976). Scheduling graphs on two processors. *SIAM Journal on Computing*, 5(1), 73–82. <https://doi.org/10.1137/0205005>
3. Fujii, M., Kasami, T., & Ninomiya, K. (1969). Optimal sequencing of two equivalent processors. *SIAM Journal on Applied Mathematics*, 17(4), 784–789. <https://doi.org/10.1137/0117070>
4. Fujii, M., Kasami, T., & Ninomiya, K. (1971). Erratum "optimal sequencing of two equivalent processors." *SIAM Journal on Applied Mathematics*, 20(1), 141–141. <https://doi.org/10.1137/0120018>
5. Tarjan, R. E. (1975). Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2), 215–225. <https://doi.org/10.1145/321879.321884>

6. Hopcroft, J. E., & Ullman, J. D. (1973). Set merging algorithms. *SIAM Journal on Computing*, 2(4), 294–303. <https://doi.org/10.1137/0202024>
7. Baker, H. (2021, July 10). *How many atoms are in the observable universe?*. LiveScience. <https://www.livescience.com/how-many-atoms-in-universe.html>
8. Kwok, Y.-K., & Ahmad, I. (1999). Static scheduling algorithms for allocating directed task graphs to multiprocessors. *ACM Computing Surveys*, 31(4), 406–471. <https://doi.org/10.1145/344588.344618>
9. Hu, T. C. (1961). Parallel sequencing and assembly line problems. *Operations Research*, 9(6), 841–848. <https://doi.org/10.1287/opre.9.6.841>
10. Papadimitriou, C. H., & Yannakakis, M. (1979). Scheduling interval-ordered tasks. *SIAM Journal on Computing*, 8(3), 405–409. <https://doi.org/10.1137/0208031>
11. Bio, M. (2021, December 31). *A brief history of the multi-core desktop CPU | techspot*. A Brief History of the Multi-Core Desktop CPU. <https://www.techspot.com/article/2363-multi-core-cpu/>
12. Kwok, Y.-K., & Ahmad, I. (1997). Efficient scheduling of arbitrary task graphs to multiprocessors using a parallel genetic algorithm. *Journal of Parallel and Distributed Computing*, 47(1), 58–77. <https://doi.org/10.1006/jpdc.1997.1395>